

반복되는 운영 업무를, 프로덕션 AI 시스템으로.

업무의 **경계와 예외**를 구조화하고, 제품 설계부터 배포·관측·운영까지 책임
집니다. 병원의 전화 업무에서 시작해 통화 운영 분석과 사내 업무지원
Agent로 문제의 범위를 넓혀 왔습니다.

Voice AI만 개발한 엔지니어 → 운영 업무를 시스템으로 옮기는 AI Product Engineer

FIVE CHAPTERS · ONE PATTERN

- 01 **아웃바운드 Voice AI**
전화 업무를 실제 전화망에서 실행
- 02 **인바운드 Voice AI**
병원별 정책을 설정 데이터로 전환
- 03 **병원 고객상담 Agent**
상담 상태와 지식을 하나의 그래프로
- 04 **통화 관측·평가**
실행 결과를 재구성 가능한 근거로
- 05 **AIU 사내 업무지원 Agent**
같은 원칙을 사내 AX 영역으로 확장

610,100+

프로덕션 AI 통화

3,927/일

일 평균 통화

295+

도입 병원

6

다국어 운영

기술이 아니라, 업무 구조에서 시작합니다

무엇을 쓸지 고르기 전에 누가 어떤 업무를 반복하고, 어디에서 판단이 흔들리며, 어떤 예외가 운영을 멈추게 하는지부터 정의합니다.

01	업무 이해	반복 구간과 사람이 병목이 되는 지점을 먼저 찾습니다.
02	경계 설계	정책·권한·예외를 코드와 설정으로 고정해 모델의 추론과 분리합니다.
03	제품화	실제 도구·데이터·채널에 연결해 운영 가능한 상태까지 만듭니다.
04	운영 개선	관측과 평가 결과를 다음 설계의 근거로 되돌립니다.

프론트엔드·백엔드·인프라 경험 위에서 Multi-Agent, 실시간 음성 처리, 검색, LLM 평가를 하나의 제품 경계로 연결해 왔습니다. PoC보다 **운영 가능한 상태와 다음 개선의 근거**를 만드는 데 집중합니다.

2025.03 – 현재	와이즈에이아이 · AI 엔지니어 / AI 프로젝트 팀장 — 병원 Voice AI 제품과 사내 업무지원 Agent 설계·개발·운영
2019.03 – 2025.02	투썸월드 · 소프트웨어 엔지니어 — 교육/AI 서비스 웹 제품 개발
2017.10 – 2018.12	투미유 · iOS & Backend — 영어회화 학습 서비스 개발
2014.12 – 2016.02	에스콜닷컴 · 웹 개발자 — 반응형 웹 빌더 · 쇼핑몰 플랫폼

PRODUCT BUILDING TOOLKIT

Agent & LLM Systems

- LiveKit Agents
- LangGraph
- Deep Agents
- LangChain
- AG-UI
- trustcall
- OpenAI Realtime
- Claude API
- Gemini 2.5 Pro

Data · Search · Evaluation

- Kafka
- Qdrant Hybrid
- PostgreSQL
- MySQL
- Redis
- RabbitMQ
- Langfuse
- LangSmith

Product & Infrastructure

- Python
- TypeScript
- Bun
- FastAPI
- Next.js
- React
- AWS ECS Fargate
- Kubernetes
- Docker
- Terraform
- SIP Trunk
- Google Cloud OKF

Open Source · LangChain-OpenTutorial Core Contributor (2025.02)

다섯 프로젝트, 하나의 반복되는 패턴

전화 한 통의 자동화에서 시작해 병원별 정책, 복합 상담, 운영 지표, 사내 지식 조회로 범위를 넓혔습니다. 대상은 달라졌지만 매년 **사람의 반복 판단을 구조화된 시스템으로 옮겼습니다.**

01

CALL EXECUTION

아웃바운드 Voice AI

예약 확인·안내 전화를 실제 전화망에서 자동 실행

02

POLICY AS DATA

인바운드 Voice AI

병원별 정책과 상담원 연결을 설정 기반 흐름으로 전환

03

STATEFUL SERVICE

병원 고객상담 Agent

검색·개인정보 수집·질문 복귀가 쉬운 상담을 상태로 관리

04

OBSERVE THE WORK

통화 관측·평가

수천 건의 결과와 품질을 재구성해 운영 개선 근거로 변환

05

INTERNAL OPERATIONS

AIU 업무지원 Agent

5개 사내 시스템의 지식과 조회 업무를 Web·Slack으로 통합

환자의 전화 업무 실행 — 운영의 관측 — 동료의 사내 업무

이 포트폴리오가 보여주는 것 · Voice AI라는 한 기술이 아니라, 업무를 이해하고 경계를 설계한 뒤 실제 운영 데이터로 개선하는 재사용 가능한 제품 개발 방식입니다.

색의 규칙 · Chapter 번호는 초록(전화 실행)에서 파랑(사내 지식)으로 이동합니다. 다이어그램 안에서는 초록이 **내가 소유·설계한 것**, 파랑이 **경계와 위임**, 노랑이 **예외와 한계**를 뜻합니다.

다섯 Chapter를 한눈에

각 Chapter는 하나의 운영 문제와 검증 가능한 설계 증거를 다룹니다. 행을 누르면 해당 Chapter의 첫 슬라이드로 이동합니다.

01	아웃바운드 Voice AI CALL EXECUTION	voxBridge 수명주기 소유 · SingleAgent와 flow_config · 검증된 예약 신청 · AMD 경계 · Kafka 분석 · 이중 배포	7 slides →
02	인바운드 Voice AI POLICY AS DATA	5종 flow node · Dynamic Booking v3 stage-before-commit · 공용 Redis FIFO Warm Transfer · 4단계 검증	4 slides →
03	병원 고객상담 Agent STATEFUL SERVICE	상태형 상담 기반 · 개인정보 수집 후 질문 복귀 · Qdrant Hybrid 지식 · 현재 Harness Runtime으로 진화	3 slides →
04	통화 관측·평가 OBSERVE THE WORK	결정론적 업무 결과 · 근거 turn 기반 Semantic QA · 개인정보 저장 경계 · 운영 대시보드	2 slides →
05	AIU / AX 업무지원 Agent INTERNAL OPERATIONS	Web-Slack 동일 런타임 · 4 specialist source isolation · 5 OKF source · SELECT-only 분석 · OpenBot 실행 경계	7 slides →

BUILD

업무 실행

전화를 실제 시스템으로

Chapter 01

CONFIGURE

정책 데이터화

병원 차이를 설정으로

Chapter 02 · 03

OBSERVE

결과 재구성

이벤트와 의미를 분리

Chapter 04

EXPAND

사내 업무로

같은 원칙을 AX에 적용

Chapter 05

— 2025 - 현재 · 1인 설계 · 개발 · 운영

병원 아웃바운드 Voice AI Agent

Sole owner

BACKGROUND

사람이 반복하던 예약 확인·안내 전화를 병원별 정책과 예외를 유지한 채 실제 전화망에서 자동화해야 했습니다. 발신 수명주기부터 예약 신청·상담 원 연결·분석까지 하나의 제품으로 묶었습니다.

PROCESS

01 · OWN

voxBridge가
Room·SIP 발신과
cleanup을 단독 소유

02 · CONTROL

SingleAgent·5종
flow
node·AgentTask로
대화과 업무 경계 분리

03 · LEARN

Transcript·Event·Re
cording을 Kafka 분석
으로 비동기 전환

OUTCOME

550,000+

아웃바운드 통화 성공

2,500/일

일 평균 실시간 통화

300+

도입 병원

MY
ROLEArchitecture · Core Implementation · Production
Operations

PROBLEM 반복 수동 전화와 결과 기록

대상 목록
예약·안내 업무

→

병원 직원
직접 발신·응대

→

수기 결과
후속 조치 기록

처리량·일관성·분석 가능성이 사람의 시간과 숙련도에 종속됩니다.

DESIGNED SYSTEM

AFTER 실행부터 분석까지 하나의 운영 시스템

**Campaign
API**
목적·병원 정책

→

voxBridge
Room · SIP
lifecycle

→

LiveKit
SIP · AMD ·
Egress

→

SingleAgent
Flow · Task ·
Transfer

EXECUTE

예약 신청 · FAQ · Warm/Cold Transfer

OBSERVE

Kafka → 전사 보완 · 결과 재구성 · QA

SIP active gate

subscribe-then-snapshot

Tokyo ECS · Korea K8s

발신과 대화의 소유권을 분리했습니다

Agent는 직접 전화를 걸지 않습니다. **voxBridge가 Room·SIP 발신과 연결 전 실패를 소유하고**, Agent는 SIP 상태를 관찰하며 대화·녹음·분석 payload만 책임집니다.

voxBridge OWNS LIFECYCLE	Room 생성 통화 단위 격리	SIP 발신 45s 계층 timeout	연결 전 실패 busy · no answer · 거부	단일 cleanup Room 정리 소유권 1곳
↓ HANDOFF - SIP.CALLSTATUS = ACTIVE				
LiveKit PLATFORM	Participant event subscribe-then-snapshot	Egress 연결 이후 녹음 시작	AMD human / machine / uncertain	Media WebRTC 오디오 경로
↓ HANDOFF - HUMAN 또는 UNCERTAIN 판정 후에만				
SingleAgent OWNS CONVERSATION	Flow node condition → exit	Tool · AgentTask 예약 · DTMF · Transfer	구조화 Event 업무 결과를 typed data로	Kafka payload transcript · event · anchor

중복 발신 방지

SIP 생성과 Room 정리 소유권을 한 곳에 두어 두 주체가 같은 Room을 정리하는 race를 없앴습니다.

Listener 뒤 Snapshot

Participant listener를 먼저 등록한 뒤 현재 상태를 다시 읽어, 이미 지나간 이벤트와의 관측 공백을 막습니다.

연결 뒤에만 부수효과

SIP active 이후에 녹음과 AMD를 시작해 연결되지 않은 통화의 잘못된 후처리를 차단합니다.

한 Agent가 맥락을, Task가 경계를 소유합니다

병원마다 Agent를 새로 만들지 않습니다. **설정이 통화의 큰 흐름을 결정하고**, 하나의 SingleAgent가 Context를 유지한 채 되돌리기 어려운 구간만 Task에 위임합니다.

flow_config · 배포 없이 조합

condition

수신자·병원 설정값으로 분기

greeting

고정 안내 · Voice / DTMF

agent

업무 대화 · 강제 Tool

action

Transfer · Log

exit

종료 안내 · Event

통화 목적도 데이터로

call_additional_guidance에 목적·사실 Context·긍정/부정 행동·금지 Tool을 넣어, 같은 응답도 통화 목적에 맞게 처리합니다.

SingleAgent · 하나의 Context

예약을 중단하고 병원 정보를 물었다가 다시 돌아와도 같은 세션과 정책 Context를 유지합니다.

Booking Tools

조회 · 신청 · 변경 요청 · 취소 요청

Bounded Tasks

생년월일 · 진료과 · 일정 · Transfer

FAQ Grounding

시작 시 기관별 활성 FAQ를 주입

Action Mode

auto · transfer · leave_memo

중요한 구분 · booking_agent와 info_agent는 별도 Agent 인스턴스가 아니라 설정 호환성을 위한 논리 노드 ID입니다.

확정 예약이 아니라, 검증된 신청을 접수합니다

LLM은 대화를 진행하지만 **후보의 존재·허용 범위·최종 동의 일치**는 코드와 **AgentTask가 검증**합니다. 통화에서 끝나는 것은 예약 확정이 아니라 신청 접수입니다.

Booking gates · 예약 신청의 결정적 경계

01 · IDENTITY

생년월일 DTMF 수집 · 낭독 확인

본인 확인 전에는 어떤 일정도 노출하지 않습니다.

02 · REFERENCE

병원 기준 진료과·의료진 목록 안에서만 선택

모델이 존재하지 않는 의료진을 만들 수 없습니다.

03 · STAGE

실시간 일정 재조회 후 정확한 후보 스테이징

검색 결과가 보였다는 사실만으로 동의를 받지 않습니다.

04 · CONSENT

동의를 한 일시와 스테이징된 일시 일치 검증

불일치하면 Commit을 차단하고 다시 확인합니다.

05 · SUBMIT

병원 확인 전 예약 신청 정보 기록·안내

최종 확정 = 병원 확인 절차가 담당한다고 명확히 안내합니다.

Staged voice pipeline · 관측 가능한 지연

단일 Speech-to-Speech 모델보다, 업무별 Tool 통제와 transcript 감사·지연 원인 분리가 중요한 환경을 선택했습니다.

AWS STT

Streaming ·
backchannel filter

→

LLM

GPT-5.4-mini ·
Gemini fallback

→

Google TTS

Streaming · 병원별
Voice/Speed

Preemptive Generation

Turn 확정 전 LLM·TTS 일부를 겹쳐
체감 대기를 줄입니다.

Provider Fallback

Primary 장애가 통화 전체 실패로 번
지지 않게 분리합니다.

Turn별로 분리해 관측하는 지연

STT delay

Turn detection

LLM TTFT

TTS TTFB

Playback

E2E

모델의 대화 능력보다 먼저, 말하지 않을 순간을 고정했습니다

잘못된 순간에 말하거나 연결하거나 종료하지 않도록 **상태 경계를 코드로 고정**했습니다. 아래 셋은 모두 운영 중 실제로 발생한 문제에서 나왔습니다.

조건 / 상황	방어 메커니즘	코드가 강제하는 것
AMD 판정 전 사람인지 기계인지 모름	FakeAgent 무발화 Agent	SIP active 뒤 LiveKit AMD를 실행하고, human 또는 uncertain 판정에서만 SingleAgent를 활성화합니다. AMD 중 시작된 turn이 Agent 전환 뒤 완료되는 race를 막기 위해 FakeAgent와 GreetingDtmfTask 양쪽에서 StopResponse를 강제하고 회귀 테스트로 고정했습니다.
사용자 무응답 듣고 있는지 알 수 없음	user_state_changed 2회 확인 후 종료	사용자 away 상태를 감지해 10초 간격으로 2회 확인하고 미응답 시 종료합니다. 단 AMD·DTMF Task·Warm Transfer 구간에서는 전역 무응답 처리를 억제해, 정상적인 대기 시간을 이탈로 오판하지 않습니다.
상담원 연결 경합 인·아웃바운드가 같은 자원 요청	공용 Redis FIFO trunk 점유권	병원별 Redis ZSET FIFO와 connected occupancy lock으로 직렬화하고, AI 브리핑 후 상담원 DTMF 수락을 받습니다. 대기 재확인·재시도·수신자 이탈을 상태로 기록하고, 번호 없음이나 연결 실패는 메모 접수로 fallback합니다.

구조화 Event 계약 · 런타임 사실을 문자열이 아닌 typed data로

name
BOOKING_CREATE · WARM_TRANSFER_CALL · CALL_TERMINATION

+

status
STARTED · STAGED · SUCCESS · FAILED · EXITED

+

details
reason · stage · mode · count · business_hours · wait_seconds

통화를 처리한 뒤, 운영이 시작됩니다

실시간 통화는 종료 이벤트와 원본 증거만 Kafka로 전달합니다. 분석 Consumer가 결과를 재구성하므로 **분석 장애가 통화에 영향을 주지 않습니다.**

실시간 통화 REALTIME PATH	대화 종료 세션 · transcript	구조화 Event extra.agent_event	녹음 anchor S3 구간 기준점	단계별 latency turn metric
--------------------------------	---------------------------------	---------------------------------------	-------------------------------	-----------------------------------

↓ PUBLISH — KAFKA 비동기 경계, 여기서 책임이 끊깁니다

분석 Consumer ASYNC PATH	02 · Restore Gemini 2.5 Pro로 상담원 연결 이후 구간 보완 전 사	03 · Classify 결정론 26개 + 의미 3개 Boolean 결과	04 · Summarize GPT-5.6 Luna · Gemini 3.5 fallback
----------------------------------	---	--	---

FACTS FROM EVENTS

실제 실행 결과는 이벤트가 결정

BOOKING_CREATE · INFO_LOOKUP · WARM_TRANSFER_CALL ·
LEAVE_MESSAGE · CALL_TERMINATION의 상태 이력으로 완료·이탈·timeout
을 재현합니다.

MEANING FROM CONVERSATION

대화에서만 알 수 있는 의미만 LLM이

trustcall은 거절·긍정·불명확 응답만 보완합니다. LLM이 구조화 이벤트가 이미 증
명한 업무 결과를 다시 쓰지 않도록 역할을 분리했습니다.

재처리 가능한 계약 · 원본 증거와 파생 결과를 분리해, 모델이나 분류 규칙이 바뀌어도 같은 통화를 다시 분석할 수 있습니다.

두 배포 경로, 하나의 운영 계약

환경은 병원 요구에 따라 달라도, 실시간 통화를 지키는 여섯 가지 계약은 동일하게 유지했습니다.

배포 경로	운영 구성	운영을 지키는 여섯 가지 계약	
Tokyo AWS ECS Fargate	시간대·부하 기반 확장 · DEV/QA/PROD · ARM64 · Multi-AZ · 프라이빗 서브넷 · 최대 20 Task	01 Lifecycle Ownership	발신·Room 정리는 voxBridge, 대화·분석 증거는 Agent 가 소유
Korea Kubernetes	voxBridge와 같은 운영 경계 · dev/qa/prod cluster · ECR 환경 태그 · rollout status 기반 배포 확인	02 Deterministic Boundary	인사말·DTMF·예약 동의처럼 되돌리기 어려운 구간은 Task와 코드가 통제
S3 통화 녹음 Kafka 분석 격리 Redis Transfer 상태		03 Provider Fallback	Primary LLM 장애를 Gemini fallback으로 흡수해 한 provider를 통화 실패점으로 만들지 않음
		04 Evidence-first Events	Agent 발화가 아니라 구조화 이벤트 상태 이력으로 업무 결과를 판정
		05 Async Isolation	Kafka로 분석 실패·재처리가 실시간 대화에 역류하지 않도록 격리
		06 Regression from Incidents	AMD handoff·DTMF·Warm Transfer 장애를 재현 가능한 테스트로 전환

— 2025 - 현재 · SELF-HOSTED LIVEKIT

병원 인바운드 Voice AI Agent

Core architect

BACKGROUND

환자가 먼저 전화하는 환경에서는 운영시간·DTMF·예약·병원 정보·상담원 연결이 한 통화에 섞입니다. 병원별 차이를 코드 분기가 아니라 검증 가능한 정책 데이터로 바꿔야 했습니다.

PROCESS

01 · CONFIGURE

5종 flow node로 운영시간·인사·업무·연결·종료 조합

02 · COMMIT

Dynamic Booking v3에서 검색·staging·명시적 동의 분리

03 · SHARE

인·아웃바운드 공용 Redis FIFO와 trunk 점유 상태 관리

OUTCOME

70,000+

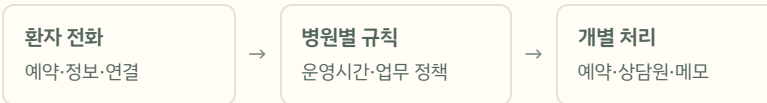
인바운드 통화 성공

1,500/일

일 평균 실시간 통화

4단계Unit → Text → Sim
→ SIP 검증MY
ROLE

Runtime Architecture · Dynamic Booking · Warm Transfer · Operations

PROBLEM 병원별 예외가 통화 경로마다 분산

정책 변경이 대화 코드와 섞이면 병원 추가·예외 처리·회귀 검증 비용이 커집니다.

DESIGNED SYSTEM

AFTER 한 Agent 위에 설정·Task·상태 머신을 분리

BOOK

기준정보 → 일정 검색 → staging → 동의

TRANSFER

Redis FIFO → trunk → briefing → connected

Client-scoped Context

Stage-before-Commit

ZSET/HASH + Stream

한 Agent와 설정이 병원별 통화를 바꿉니다

SingleAgent가 통화 전체 Context를 유지하고, **flow_config**가 운영시간·DTMF·업무 진입·연결·종료를 코드 배포 없이 조합합니다.

condition

병원 설정·운영 상태에 따른 분기

greeting

녹취 고지 · Voice / DTMF

agent

업무 요청 전달 · Tool 진입

action

Transfer · Direct · Log

exit

종료 안내 · 구조화 Event

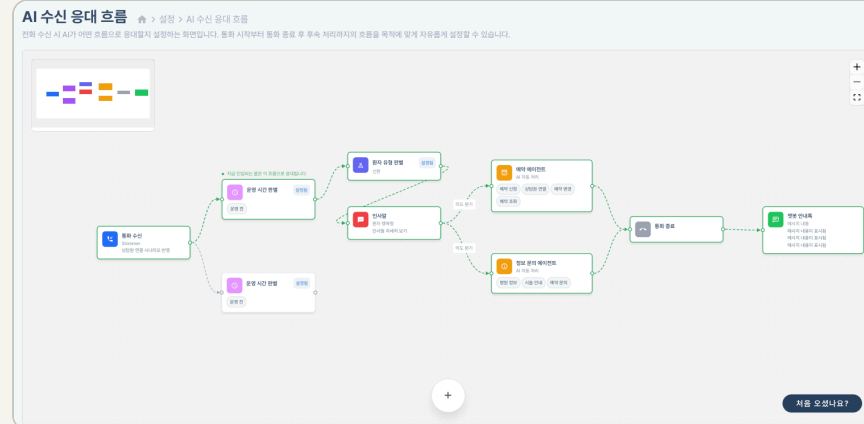
Client-scoped Call Context

시작 시 압축된 병원 근거를 주입하고, 조회에 실패하면 아는 척하지 않는 no-knowledge 상태로 통화를 계속합니다.

논리 노드 ≠ Agent handoff · booking_agent와 info_agent는 별도 Agent 인스턴스가 아니라 SingleAgent 안의 설정 호환 ID입니다.

AI 수신 응대 흐름 · 실제 설정 화면

AGENT ADMIN



CONFIG 통화 수신 → 운영시간 판별 → 인사말 → 업무 진입까지, 병원 담당자가 노드 그래프로 편집합니다.

검색과 동의를 분리한 Dynamic Booking v3

환자의 자연어 선호를 병원 기준정보와 실시간 일정의 교차점에서 검증해, **정확한 후보 하나만 동의 대상**으로 만듭니다. 대화는 LLM이, 사실과 상태는 코드가 책임집니다.

- 01 · RESOLVE
환자 식별 · 선택적 보험 확인
- 02 · REFERENCE
허용된 진료과·의료진 ID만 주입
- 03 · SEARCH
기간·요일·시간·의료진 조건으로 일정 조회
- 04 · STAGE
실시간 재검증 후 유일한 확인 문구 생성
- 05 · COMMIT
명시적 동의와 staging 일치 시 신청 기록

병원별 Action Mode

auto

AI 처리

transfer

사람 연결

leave_memo

메모 접수

네 가지 안전 불변식

- | | | |
|----|-------------------------------|---|
| 01 | Search ≠ Consent | 일정이 보였다는 사실만으로 확인 질문이나 Commit을 허용하지 않습니다. |
| 02 | Exact Stage | 진료과·의료진·일시를 함께 고정하고, 새 후보가 오면 이전 staging을 덮습니다. |
| 03 | Explicit Confirmation | 되묻거나 잘못 들은 “네?”를 동의로 처리하지 않고 staging mismatch를 차단합니다. |
| 04 | Request, not EMR Final | 통화 안의 Commit은 완전한 예약 신청 정보 기록이며 병원 최종 확정 아닙니다. |

검색 결과만으로는 질문을 만들지 않습니다 · 의료진을 임의로 확정하지 못하게 하고, 동의 일시가 현재 스테이징과 다르면 Commit 자체를 차단합니다.

상담원 연결은 병원별 공용 상태 머신입니다

한 건의 SIP 연결보다 어려운 것은 여러 환자가 같은 상담원과 trunk를 요청하는 순간입니다. **인바운드·아웃바운드 호출을 하나의 FIFO와 점유권으로 직렬화**했습니다.



세 저장 구조의 역할 분리

ZSET · Source of truth

병원별 활성 FIFO 순서와 rank 0 점유권

HASH · Live read model

상태 · 시각 · 시도 횟수 · 브리핑 · 통화 방향

STREAM · Notification

PII 없이 상태 변경만 알리고 snapshot을 다시 읽게 함

연결 성공만큼 중요한 실패 복구

- AI가 상담원에게 통화 맥락을 **브리핑**한 뒤 DTMF 1/2로 수락·거절
- 일부 timeout·error·voicemail은 최대 **3회 재시도**
- 120 · 240 · 360초에 **대기 지속 여부 재확인**
- 수신자 이탈 · queue timeout · declined를 **별도 상태로 기록**
- 연결 번호 없음이나 실패 시 **leave-message fallback**
- 상담 종료 전까지 connected 호출이 **trunk**를 점유

운영 장애를 회귀 테스트로 바꾸는 4단계 검증



— 2025.03 - 2026.03 · LANGGRAPH

병원 고객상담 AI Agent

BACKGROUND

병원 상담은 FAQ 검색만으로 끝나지 않습니다. 질문 중 개인정보 수집·예약·상담원 연결 절차가 끼어들면 사용자가 같은 맥락을 반복해야 했고, 검색 결과와 상담 상태를 하나의 Thread에서 관리할 기반이 필요했습니다.

PROCESS

01 · REMEMBER

pending_question·collected_info·current_node를 Graph state로 관리

02 · GROUND

병원 Fact Data를 Qdrant Dense+Sparse Knowledge Tool로 연결

03 · SERVE

Thread 기반 FastAPI·LangGraph API·Docker 서비스 경계 구축

OUTCOME

질문 복귀

개인정보 절차 후 원래 상담 재개

병원별 지식

기관·의료진·시설·증명서 Tool

확장 기반

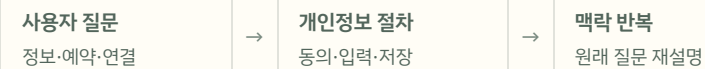
Harness · Voice · Kakao 진화 수용

MY ROLE

Initial Graph · State/Tool Flow · Knowledge · API Boundary

Foundation architect

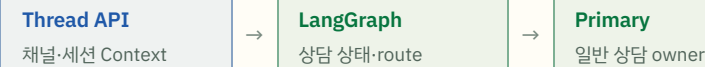
PROBLEM 절차 전환 때 상담 맥락이 끊김



상담·개인정보·검색 결과가 분리되면 대화 경험과 후속 실행 모두 불안정해집니다.

DESIGNED SYSTEM

AFTER 질문·절차·근거를 공유하는 Stateful Graph



COLLECT

동의 → 추출 → 검증 → 저장 → 질문 복귀

GROUND

Qdrant Hybrid Search → typed Knowledge Tool

Postgres · Redis

Docker service boundary

Team-added Harness guard

상태형 상담 기반이 Harness Runtime으로 진화했습니다

현재 Graph는 내가 설계한 초기 상담 노드를 **team wrapper**로 감싸고 **current_node legacy** 값을 유지합니다. 기반을 버리지 않고 통제 계층을 추가한 것이 구조적 확장성의 증거입니다.

2025.03 — 내가 설계·구현한 기반

primary_assistant

일반 상담 · 병원별 Knowledge Tool

customer_interaction

동의 · 개인정보 수집

tools

Qdrant Hybrid Search

extract_personal_info

이름 · 전화번호 · 생년월일 검증·저장

pending_question

collected_info

is_personal_info_saved

current_node

FastAPI

Gateway ·
Thread

LangGraph API

State · Run

Postgres · Redis

Persistence

이후 팀이 확장한 현재 Runtime

START → harness_agent

입력 튜닝 · 언어 정규화 · route_entry

general consultation

일반 상담 팀

required personal info

필수 정보 팀

customer info collection

추출·저장 팀

tool coordination

지식 Tool 팀

harness_controller

허용 전이 whitelist · Tool loop suspicion · route guard

fast_safe / full

위험도별 controller 경유

Channel Guard

Kakao 버튼 · 이미지 · 다국어

초록 노드는 내가 만든 상담 노드가 team wrapper 안에서 그대로 살아 있는 부분입니다.

상담 절차가 끼어들어도, 대화의 목적은 사라지지 않습니다

진료시간 질문과 상담원 연결이 한 문장에 섞인 요청을 실제 개발 서비스에서 검증했습니다. **먼저 답할 수 있는 정보는 제공하고**, 연결에 필요한 정보만 수집한 뒤 같은 Thread에서 신청을 완료합니다.

정보 응답 → 개인정보 수집

01 · BRANCH

<
와이즈병원 챗봇
<

진료시간을 안내드립니다.
월, 화, 목, 금: 오전 07:00 ~ 오후 07:30까지입니다 (점심시간은 오후 01:00~ 오후 02:00)
수: 오전 09:00 ~ 오후 06:30 까지이고요 (점심시간은 오후 01:00~ 오후 02:00)
토: 오전 07:00 ~ 오후 02:00 까지입니다.

상담 연계를 위한 개인정보 수집·이용에 동의하는 경우 아래 정보를 입력해 주세요.

개인정보를 입력하지 않아도 챗봇은 이용할 수 있어요.

ANSWER 진료시간 답변을 먼저 완결하고 이름·전화번호·생년월일만 요청합니다.

동의 확인 → 상담 신청 완료

02 · RESUME

<
와이즈병원 챗봇
<

예시:
홍길동 01012345678 19900115
개인정보 처리방침 확인하기

홍길동 01012345678 19900115

개인정보 수집·이용에 동의해 주셔서 감사합니다

RESUME 예시 데이터를 확인한 뒤 질문을 다시 설명하게 하지 않고 연결 신청을 완료합니다.

State · pending question과 수집 정보를 Thread에 유지

Boundary · 개인정보 미입력 상태에서도 일반 상담 가능

Evidence · 공개 개발 환경, 화면의 값은 예시 데이터

— 2026 · ANALYTICS V2 · PRIVACY-SAFE LLM EVALUATION

Voice AI 통화 관측·평가 파이프라인

Pipeline owner

BACKGROUND

통화가 하루 수천 건씩 쌓이자 사람이 녹취를 다시 듣고 결과를 판정하는 일도 반복 업무가 됐습니다. 실시간 통화와 분석 부하를 분리하면서도 한 통화의 업무 결과와 품질을 재구성해야 했습니다.

PROCESS

01 · DECOUPLE

Transcript·Agent
event·Recording·Latency를 Kafka로 비동기 전달

02 · RECONSTRUCT

결정론적 event를 시도와 통화별 최종 resolution으로 정규화

03 · EVALUATE

근거 turn을 포함한 Semantic QA와 버전 점수 발행

OUTCOME

Analytics v2

KPI에서 책임별 손실까지 drill-down

재평가 가능

room·평가 버전 기반 역등 처리

Privacy-safe

민감 원문은 로컬, 집계만 외부

MY ROLE

Pipeline Architecture · Result Normalization · LLM Evaluation · Privacy Boundary

PROBLEM 운영자가 통화 원문을 직접 확인

실시간 통화

대화·Tool·Transfer

→

녹취·로그

서로 다른 신호

→

수동 판정

결과·품질 확인

분석 장애가 실시간 경로에 영향을 줄 수 있고, 같은 통화도 평가 기준에 따라 결과가 달라질 수 있습니다.

DESIGNED SYSTEM

AFTER

실행 신호와 의미 평가를 분리한 후처리

Kafka

비동기 event stream

→

Consumer

역등 · 재처리

→

Analytics v2

attempt · resolution

FACTS

예약·연결·종료를 구조화 event로 결정

MEANING

의도·라우팅·응답 품질을 근거 turn으로 평가

Gemini 보완 전사

Langfuse versioned score

PII local boundary

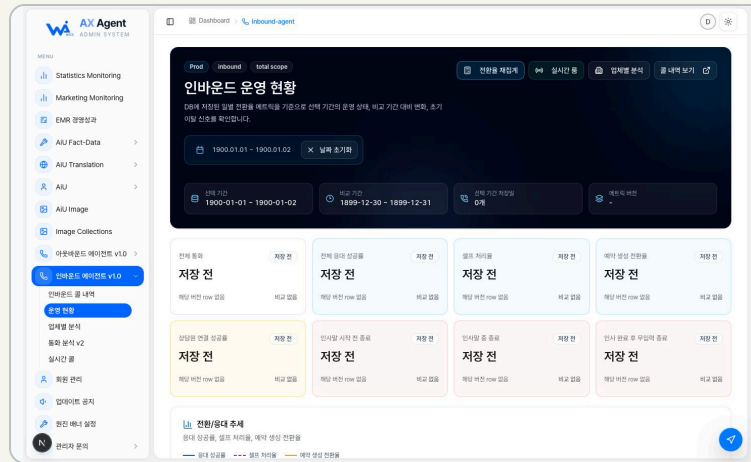
운영 성과에서 손실 원인까지, 두 개의 해상도로 봅니다

운영 현황은 기간별 성과와 추세를 빠르게 확인하고, 통화 분석 v2는 **결과 분포와 intent attempt 단위 funnel**을 내려가며 손실 지점을 진단합니다.

인바운드 운영 현황

MONITOR 인바운드 통화 분석 v2

DIAGNOSE



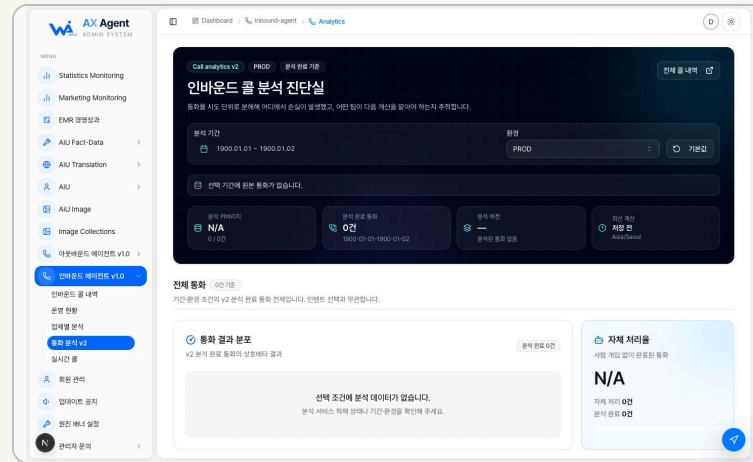
OVERVIEW 통화량·응대 성공·셀프 처리·예약·Transfer·초기 이탈을 기간 단위로 비교합니다.

Facts · 완료 여부는 구조화 Agent event로 재구성

Meaning · 품질 평가는 근거 turn과 평가 버전을 유지

Privacy · 화면은 매뉴얼의 빈 기간 예시이며 현재 운영 수치가 아님

다음 확장 · 환자 업무를 실행하고 평가한 경험은, 동료가 운영 정보와 지식을 찾는 반복 업무를 자동화하는 AIU로 이어졌습니다.



DIAGNOSE 결과 분포·자체 처리율·책임별 손실·intent funnel과 Semantic 진단 후보를 추적합니다.

— 2026.08 - 현재 · BUN · DEEP AGENTS

사내 업무지원 Multi-Agent (AIU / AX)

Architecture & build

BACKGROUND

AIU 제품과 통화 운영 정보를 5개 시스템·매뉴얼·화면에서 찾아야 했습니다. 동료의 반복 조회를 줄이되, 지식 접근·DB 조회·브라우저 실행 권한을 Agent의 추론과 분리해야 했습니다.

PROCESS

01 · UNIFY

5개 source를 OKF v0.2 catalog와 read-only backend로 통합

02 · DELEGATE

Supervisor가 4개 stateless specialist에 완전한 작업을 위임

03 · GOVERN

AG·UI·SELECT-only DB·OpenBot Tool ownership으로 실행 경계 유지

OUTCOME

약 3주

착수 후 1차 운영 가능 상태

322개

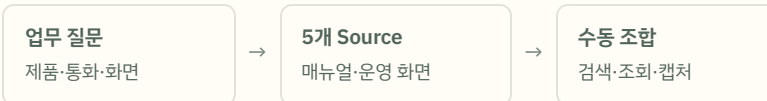
Manual · Playbook · Policy 통합

45개

Unit · Contract Test

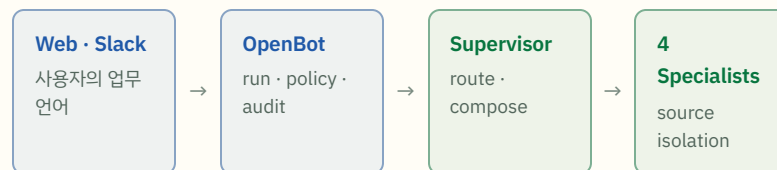
**MY
ROLE**

Architecture · Migration · Knowledge Contract · Safe Tools · Web/Slack Runtime

PROBLEM 사람이 여러 시스템에서 근거를 조합

정보 위치와 실행 권한이 흩어져 있어 질문마다 화면 탐색과 근거 확인을 반복합니다.

DESIGNED SYSTEM

AFTER Web·Slack 하나의 접점, 분리된 권한 경계

KNOWLEDGE

OKF catalog → 승인 문서·screenshot

OPERATIONS

SELECT-only MySQL → 집계·bounded QA

5 source contracts

AG·UI final boundary

Short-lived attachment

업무 화면을 찾는 대신, 업무 언어로 묻습니다

같은 자연어라도 **필요한 근거와 실행 경계는 다릅니다**. Supervisor가 제품·데이터·화면 요청을 구분하고 해당 specialist에 완전한 작업 설명을 전달합니다.

업무 도메인	동료가 실제로 던지는 질문	필요한 근거와 경계
Inbound 받은 전화는 어떻게 끝났나	“어제 예약 전환율과 인사말 전 이탈을 비교해 줘.”	저장 통화 지표는 SELECT-only 집계로, 통화 내용 확인은 비식별 bounded QA로만 접근합니다.
Outbound 발신 결과는 무엇이 달랐나	“이번 주 응답 여부와 예약 신청 전환을 병원별로 보여줘.”	응답/무응답 filter와 명시적 기간 집계만 허용하고, raw payload는 결과에서 제외합니다.
Agent Admin 설정은 어디서 확인하나	“운영시간 설정 화면과 변경 전 확인 정책을 알려줘.”	승인된 매뉴얼 문서와 catalog가 허용한 screenshot만 근거로 사용합니다.
HQ · SVC 운영 정책의 근거는 무엇인가	“이 캠페인 설정과 상담 이력 화면의 근거 문서를 보여줘.”	Policy·State Model 문서를 근거로 답하고, 현재 관찰과 문서 근거를 구분해 표시합니다.



채널은 달라도, 지식과 실행 경계는 하나입니다

“운영 현황과 통화 분석 화면의 차이를 근거와 함께 설명해 달라”는 **동일 질문을 Web과 Slack에서 실행**했습니다. 두 채널 모두 같은 Agent Admin 지식과 승인된 매뉴얼 screenshot을 사용합니다.

OpenBot Web

STRUCTURED ANSWER

Slack Thread

SAME GROUNDING

AIU Agent

Agent Admin에서 인바운드 분석 화면과 운영 현황 화면의 차이를 매뉴얼 근거와 함께 설명해줘. 관련 매뉴얼 스크린샷도 첨부해줘.

핵심 차이

구분	인바운드 운영 현황	인바운드 통화 분석 v2
목적	전체 인바운드 운영 성과와 기간별 추세 모니터링	통화 결과와 이벤트별 차이를 분석해 손실 원인 진단
매뉴얼 위치	인바운드 에이전트 v1.0 > 운영 현황 / dashboard / inbound-agent	인바운드 에이전트 v1.0 > 통화 분석 v2 / dashboard / inbound-agent / analytics
주요 정보	전체 통화, 최대 상담, 셀프 처리, 예약 생성, Transfer, 초기 이월, KPI 추세, 주력 이슈	분석 카테고리, 결과 분포, 자체 처리율, 책임 분 손실, 이벤트별 시도 완수율, 과잉, Semantic 오류인 통발
기본 분석 단위	지정된 일별 지표와 기간 집계	통화 단위 결과 및 이벤트 시도(attempt) 단위 리얼
필터	날짜 범위와 직원 동일 기간 비교	날짜, 환경(env), 이벤트, Semantic 버전
제한 사항	"기간별 운영 성과가 올라갔는가?"	"어떤 결과에서 손실이 발생했고, 어느 이벤트가 원인인가?"

1. 운영 현황 화면

운영 현황은 운영 모니터링용 요약 화면입니다. 선택한 기간의 일별 지표와 직원 기간을 비교하면서 다음과 같은 전체 성과를 확인합니다.

- 전체 인바운드 통화량
- 원인 분류

Ask anything

슬랙드

dan 1분 전
Agent Admin에서 인바운드 분석 화면과 운영 현황 화면의 차이를 매뉴얼 근거와 함께 설명해줘. 관련 매뉴얼 스크린샷도 첨부해줘.

기간별 운영 현황

Agent Admin: 인바운드 분석

구분	인바운드 운영 현황	인바운드 통화 분석 v2
목적	전체 운영 성과와 추세를 빠르게 점검	통화 결과와 원인 분석 지원
주요 데이터	일별 운영 지표	통화 결과 및 intent 시도 분석
주요 지표	전체 통화, 최대 상담, 셀프 처리, 예약 생성, transfer, 초기이월, KPI 추세	분석 카테고리, 결과 분포, 자체 처리율, 책임 분 손실, 이벤트별 시도 완수율, 과잉, Semantic 오류인 통발
분석 단위	일별 지표와 기간 집계	통화 단위 결과 및 intent 시도(attempt) 단위 리얼
필터링	날짜 범위, 직원 동일 기간 비교, 일지별 통, 통 목록-실시간 통 등으로 자동	날짜, 환경(env), intent, semantic 버전
제한 사항	"전체 운영 성과가 어떤가?"를 확인할 때	"어떤 결과가 발생했는가?"를 ?

1. 인바운드 운영 현황

운영 현황은 지정한 일별 metrics를 바탕으로 기간별 통찰력과 최대-셀프 처리-예약-transfer-초기이월 추세를 확인하는 관리를 위한 인터페이스로 지정한 기간을 비교하고, KPI 추의 신호-일지별 데이터를 확인할 수 있습니다.

2. 인바운드 통화 분석 v2

통화 분석 v2는 분석 카테고리 결과 분포를 바탕으로 자체 처리율, 책임 분 손실, intent 시도 결과와 funnel 완수율을 진단하는 분석 인터페이스. semantic 오류인 통발로 해결하지만, 이는 해당 항목이 아니라 추가 확인이 필요한 원인 때문입니다.

모든 통과 결과는 call 전체, intent funnel은 attempt 단위로 이루어져 두 지표를 단순히 분석하면 안 됩니다.

공통 참고사항

- 주 화면 모두 매뉴얼상 (inbound-agent, [v1.0, v2.0]) 권한이 필요합니다.
- 대형 역할은 admin/supervisor-manager입니다.
- 운영 현황의 '일별' 데이터는 '일별 preview' HTML, 승인 후 실행되는 변경 작업이며, 매뉴얼을 확인하는 것만으로 실행되는 경우
- 일일통 통화로 소관담당을 담당하는 기간 상에서 데이터로, 전체 운영 수치를 파악하면 됩니다.

근거: "인바운드 운영 현황", "인바운드 통화 분석 v2", "Role to screen maps", "Screenshot coverage", "Agent Admin AI 업무 매뉴얼", "인바운드 통화 분석 v2"

WEB 비교 표와 상세 설명을 넓은 화면에 구조화하고 근거 이미지를 대화에 연결합니다.

SLACK 같은 근거를 Slack 표와 파일 첨부로 변환하며 다른 채널·구성원 정보의 캡처에서 제외했습니다.

Same question · 제품 화면 차이와 근거 요청

Same runtime · Supervisor → Agent Admin specialist

Surface-owned delivery · Web attachment / Slack file

Supervisor는 답을 만들기보다 경계를 지킵니다

Supervisor는 원본 source를 읽지 못합니다. 제품·데이터 경계는 각 specialist에 선언된 source path와 Tool로만 열립니다.

Deny · Supervisor는 원본을 읽지 않음

AIU Supervisor

제품·데이터·화면 구분 · 최종 답변 조합

- /knowledge/sources read **deny**
- 메시지 텍스트로 권한을 만들 수 없음

Allow · 선언된 source와 Tool만

Inbound Agent

저장 통화 · 전환율 · QA

Outbound Agent

응답 결과 · 예약 전환

Agent Admin Agent

매뉴얼 · 화면 · 변경 정책

HQ/SVC Agent

운영 정책 · HQ/SVC UI

4개 모두 stateless — Supervisor가 매번 완전한 작업 설명을 전달합니다.

AG-UI가 내부 작업과 surface를 분리

OpenBot Run

verified run context · Web / Slack surface

Deep Agent Event Stream

하위 Agent event와 JSON result는 내부에서만 처리 · surface에 노출하지 않음

Supervisor Final

answer · evidence를 조합하고 attachment metadata를 검증

Surface Boundary

최종 text 또는 OpenBot이 실행할 Tool call만 노출

Surface Tool

호출권을 Web/Slack OpenBot으로
반환

Deployment Tool

active run + callback credential
확인

322개 문서를 한 폴더가 아닌 다섯 계약으로

각 source는 별도의 snapshot·기본 접근 metadata·문서 수·entrypoint를 갖습니다. **Catalog**는 모델이 읽기 전에 **manifest**와 **실제 파일을 대조**합니다.

OKF v0.2 source · Markdown 문서 수

AIU SVC hospital-facing UI		133
Agent Admin screen · auth · policy		62
AIU HQ head-office operations		51
Inbound Agent runtime · booking · transfer		43
Outbound Agent call flow · operations		33

사용자 매뉴얼과 AI용 **Playbook·Policy·State Model**을 하나의 OKF v0.2 번들로 통합하고, read-only virtual backend로만 노출했습니다.

Catalog가 시작 시 검증하는 것

- 01 Source Manifest** — source_id · snapshot · endpoint
- 02 Document Count** — 선언 수와 실제 Markdown 파일 수 일치
- 03 Frontmatter** — classification · roles · claims
- 04 Screenshot Path** — 문서가 참조하는 화면 자산 존재 여부

코드로 강제되는 현재 경계

- /knowledge는 write가 **항상 deny**
- Supervisor는 모든 source 원문 read가 **deny**
- Specialist는 자신의 source path만 read
- Screenshot은 catalog가 승인한 문서·index만 반환

정직한 현재 범위 · verified AIU caller는 아직 동일 full-access identity를 사용합니다.
Frontmatter의 role/claim은 계약 metadata이며 사용자별 RBAC 완성으로 표현하지 않습니다.

운영 데이터를 읽되, 조회 범위는 코드로 고정했습니다

자연어 질문이 데이터베이스에 닿기까지 **네 개의 축소 관문**을 지납니다. 각 관문에서 무엇이 제거되는지가 이 Tool의 계약입니다.

요청 축소 NARROWING	Natural language 날짜 · 병원 · 환경 · 지표	Zod boundary 정확한 날짜 · enum · 상호배타 flag	Bound SELECT parameter binding · SSL CA 인증서 검증	Safe result 집계 · 근거 label · hard limit
결과에서 제외 STRIPPED	이름 기본 결과에 미포함	전화번호 기본 결과에 미포함	Transcript 기본 결과에 미포함	녹음 · Raw payload 기본 결과에 미포함

일상 운영 조회

Inbound 저장 통화 검색 · 자체 처리율 · 예약 전환 · 상담원 연결 · 일별 운영 metric
Outbound 응답/무응답 filter · 통화 결과 · 예약 신청 전환 · 명시적 기간 집계

민감 통화 Deep Review

명시적으로 선택한 건만

최대 5건
명시적 record id

최대 120 turn
앞·뒤 bounded view

- 이름·전화번호·패턴형 식별자 redaction
- transcript를 **untrusted data**로 취급
- evidence turn index와 limitation만 유지
- 최종 답변에 raw transcript·개인정보 재노출 금지

Fail-closed by configuration · SELECT-only 계정이 없으면 지식과 Computer Use는 계속 동작하지만, DB Tool은 명확한 설정 오류를 반환합니다.

실행 권한과 전달 경계를 OpenBot에 남겼습니다

AIU Agent는 reasoning과 위임을 담당하지만 브라우저·plugin·surface Tool의 실행 주체가 아닙니다. OpenBot이 검증된 run, 사용자 surface와 실행 정책을 계속 소유합니다.

경계	계약	강제 방식
Admission	OpenBot Run Assertion	AG-UI 요청에 run context와 aiu-agent Bot ID가 없으면 401로 거부합니다.
Execution	Tool Ownership	Surface Tool은 OpenBot UI로 호출권을 반환하고, deployment Tool은 active run과 callback credential을 확인한 뒤 OpenBot이 실행합니다.
Computer Use	Policy · HITL · Audit	브라우저와 plugin 권한, secret 입력과 사람 승인은 OpenBot 정책 경계에 남깁니다.
Delivery	Short-lived Attachment	매뉴얼 screenshot은 HMAC URL, 생성 이미지는 난수 URL로 대화 상태 밖에서 전달합니다.

1차 범위에서 완료 · 약 3주

- Deep Agents 마이그레이션과 AG-UI Web·Slack 런타임
- 5 source OKF catalog와 specialist source isolation
- SELECT-only 운영 조회 · 최대 5건 비식별 QA
- OpenBot Tool ownership·첨부 경계 · 45개 AIU test

아직 말하지 않는 것

절감 시간

전사 확산률

사용자별 RBAC

- 측정하지 않은 성과는 수치로 주장하지 않습니다
- verified caller별 세밀한 role/claim 집행은 미완성
- DB 계정이 없을 때의 운영 분석 실행
- 세일즈·재무·HR 전사 업무 자동화

업무를 이해하고, 경계를 설계하고, 운영으로 증명합니다.

Voice AI, 통화 관측·평가, 사내 업무지원 Agent까지. 기술이 달라져도 반복 업무를 프로덕션 AI 시스템으로 바꾸는 방식은 같습니다.

Workflow Structuring

Multi-Agent Systems

Observability & Evaluation

Production Operations

End-to-End Ownership

다섯 프로젝트에서 반복된 네 단계

01 · BUILD

업무 실행

Ch 01

사람이 반복하던 실행을 실제 시스템으로 옮깁니다.

02 · CONFIGURE

정책 데이터화

Ch 02 · 03

기관별 차이와 예외를 코드가 아닌 설정으로 고정합니다.

03 · OBSERVE

결과 재구성

Ch 04

실행 사실과 의미 평가를 분리해 다시 계산할 수 있게 합니다.

04 · EXPAND

다음 업무로

Ch 05

같은 경계 설계를 새로운 업무 영역에 적용합니다.

 kofsitho@naver.com GitHub LinkedIn Tech Blog Medium